



SubSync

Transcription & sous-titrage vidéo / audio

FastAPI

Celery

Redis

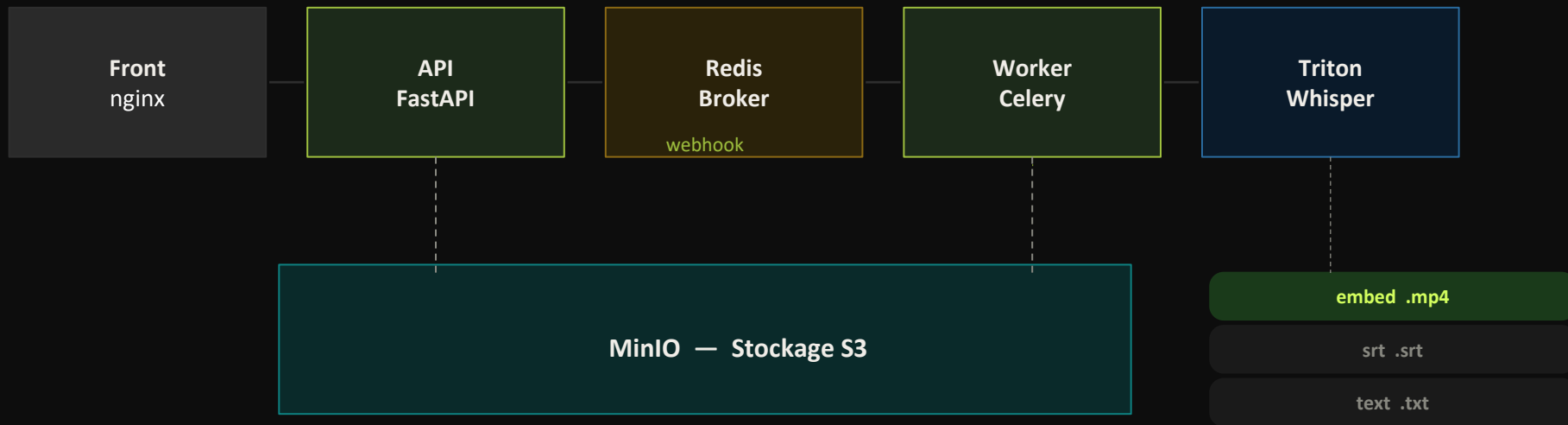
MinIO

Triton

Whisper

Vue d'ensemble

Pipeline de bout en bout



Audio / Vidéo

Résultat (presigned URL)

FastAPI

Couche API & événements



ASGI / async natif

Gestion de milliers de connexions SSE simultanées sans blocage.



Validation Pydantic

Schémas stricts sur les entrées/sorties, erreurs HTTP lisibles.



Server-Sent Events

StreamingResponse pour pousser le résultat au client dès réception du webhook.



Docs auto /docs

Swagger UI généré automatiquement, pratique pour tester les endpoints.

Endpoints clés

POST /subtitles

GET /subtitles/{id}/stream (SSE)

GET /subtitles/{id} (poll)

DELETE /subtitles/{id}

POST /internal/webhook

Celery

File de tâches asynchrone

- **Découplage API / Worker**

L'API répond en < 50ms et délègue la transcription (30s–5min) au worker sans bloquer.

- **Retry automatique**

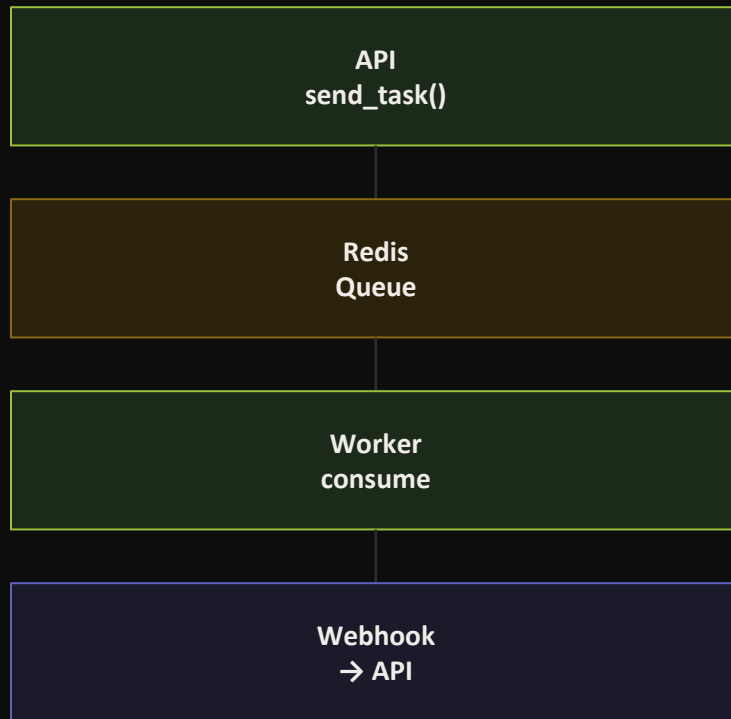
En cas d'échec Triton ou réseau, la tâche est relancée avec backoff configurable.

- **Scalabilité horizontale**

Plusieurs workers peuvent tourner en parallèle sans modifier l'API.

- **État persistant**

Les résultats sont stockés dans Redis (backend) pour le polling /subtitles/{id}.



Redis & MinIO

Broker de messages et stockage objet éphémère

Redis

- **Broker Celery**

Pub/sub des tâches JSON (clé S3 + langue + mode + callback).

- **Backend résultats**

Stocke les états PENDING → STARTED → SUCCESS/FAILURE.

- **Bus SSE in-memory**

asyncio.Queue par job_id pour pousser le résultat webhook → client SSE.

- **Légèreté**

Redis ne transporte que des métadonnées (< 1 KB), jamais de binaire audio.

MinIO

- **Compatible S3**

API AWS S3 identique — migration vers S3, GCS ou Azure Blob sans changer le code.

- **Expiration 24h**

ILM rule sur le bucket : fichiers source supprimés automatiquement après 1 jour.

- **Découplage taille**

Binaires vidéo (> 100 MB) transférés en dehors de Redis pour ne pas saturer le broker.

- **Presigned URLs**

Le client télécharge directement depuis MinIO, sans passer par l'API.

Triton Inference Server

NVIDIA — serveur d'inférence hautes performances



Dynamic Batching

Regroupe les requêtes d'inférence simultanées en un seul batch GPU, maximisant l'utilisation du GPU même sous forte charge.



Model Repository

Gère plusieurs modèles (Whisper tiny/base/large) sans redémarrer le serveur. Versionning natif.



Protocole standard

API HTTP/gRPC KServe v2. Compatible avec n'importe quel client Python via tritonclient.



Métriques Prometheus

Expose /metrics nativement : latence, queue depth, GPU utilization — scraped par Prometheus.



Backend agnostique

Supporte PyTorch, ONNX, TensorRT, Python custom backend. Whisper tourne en Python backend.



Sequence Batching

Pour les modèles stateful (futur : streaming Whisper), gestion des sessions avec correlation ID.

Whisper & Dynamic Batching

Transcription précise avec gestion des longs fichiers

● Modèle

OpenAI Whisper — transformer encoder-decoder entraîné sur 680k heures audio multilingues.

● Détection langue

Langue détectée automatiquement sur les 30 premières secondes si non spécifiée.

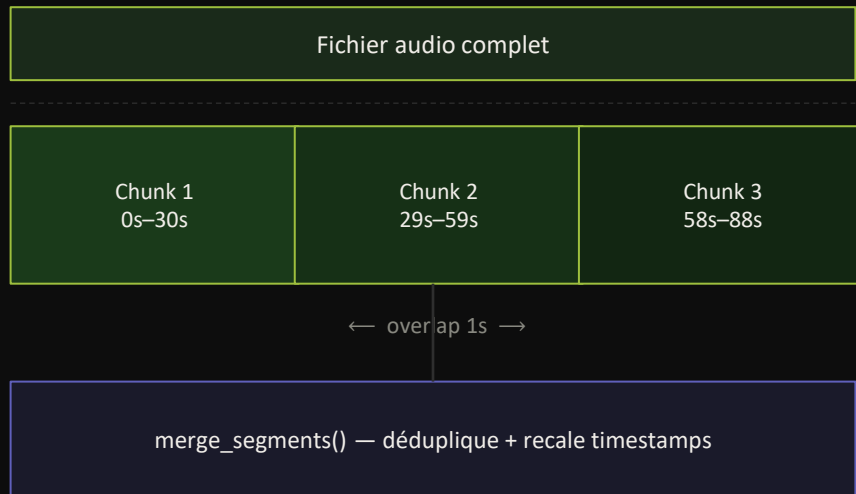
● Sorties Triton

Retourne segments JSON avec start / end / text pour chaque phrase détectée.

● Tâches

transcribe (texte original) ou translate (traduit vers l'anglais).

Découpage audio (chunking)



Segments finaux horodatés

Formats de sortie

Trois modes selon le besoin — vidéo ou audio en entrée



embed

Vidéo sous-titrée

Sous-titres intégrés comme piste soft (sans ré-encodage). Copie des flux audio/vidéo avec ffmpeg -c copy.

Sortie : *vidéo_subtitled.mp4*

- Codec adapté au conteneur : mov_text (mp4/mov), srt (mkv/avi), webvtt (webm).
- Vidéo uniquement — incompatible avec les fichiers audio.



srt

Fichier SRT

Fichier de sous-titres standard avec timestamps précis au millième de seconde.

Sortie : *fichier.srt*

- Format HH:MM:SS,mmm → HH:MM:SS,mmm compatible VLC, Premiere, DaVinci.
- Vidéo et audio.



text

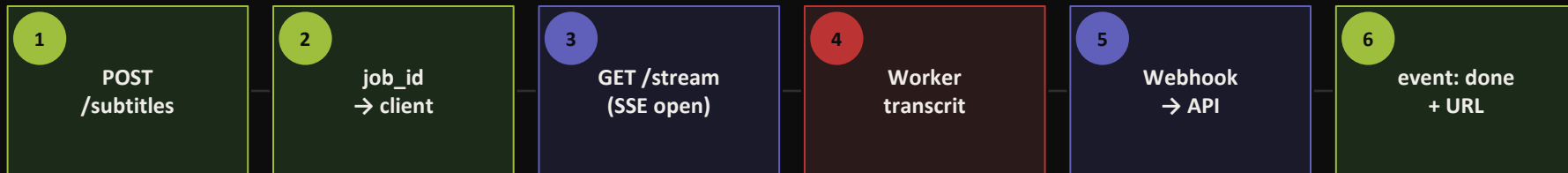
Texte brut

Transcription pure sans timestamps, un segment par ligne. Idéal pour du traitement NLP ou compte-rendu.

Sortie : *fichier.txt*

- Directement copiable depuis l'interface.
- Vidéo et audio.

Notification temps réel — SSE



Timeout SSE : 30 min — 1 connexion par job — `asyncio.Queue(maxsize=1)` libérée après livraison

SSE (choix retenu)

- Connexion unique, résultat poussé dès dispo.
- 0 requête inutile → économie réseau.
- Latence minimale côté client.

Polling classique (fallback)

- GET /subtitles/{id} disponible aussi.
- Utile pour clients sans support SSE.
- Overhead : N requêtes × intervalle.



Stack complète

Chaque brique choisie pour une raison précise

FastAPI	API async + SSE ASGI natif, validation Pydantic, docs auto	Celery	Worker asynchrone Découplage, retry, scalabilité horizontale
Redis	Broker + backend Ultra-rapide, pas de binaire, état centralisé	MinIO	Stockage S3 Compatible AWS S3, TTL auto, presigned URLs
Triton	Inférence GPU Dynamic batching, multi-modèle, métriques	Whisper	Modèle de transcription SOTA multilingue, segments horodatés
ffmpeg	Audio/vidéo processing Extraction audio, embedding sous-titres soft	Prometheus + Grafana	Observabilité Métriques GPU, latence Triton, worker health